

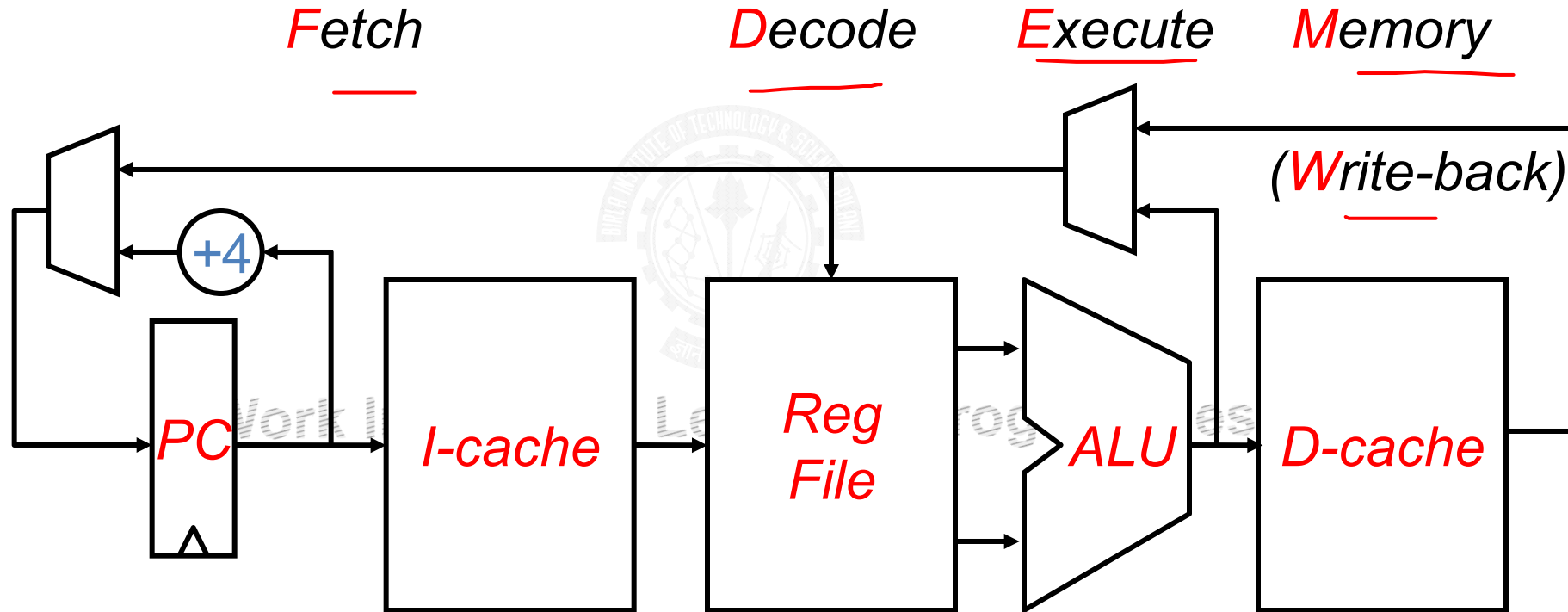


BITS Pilani

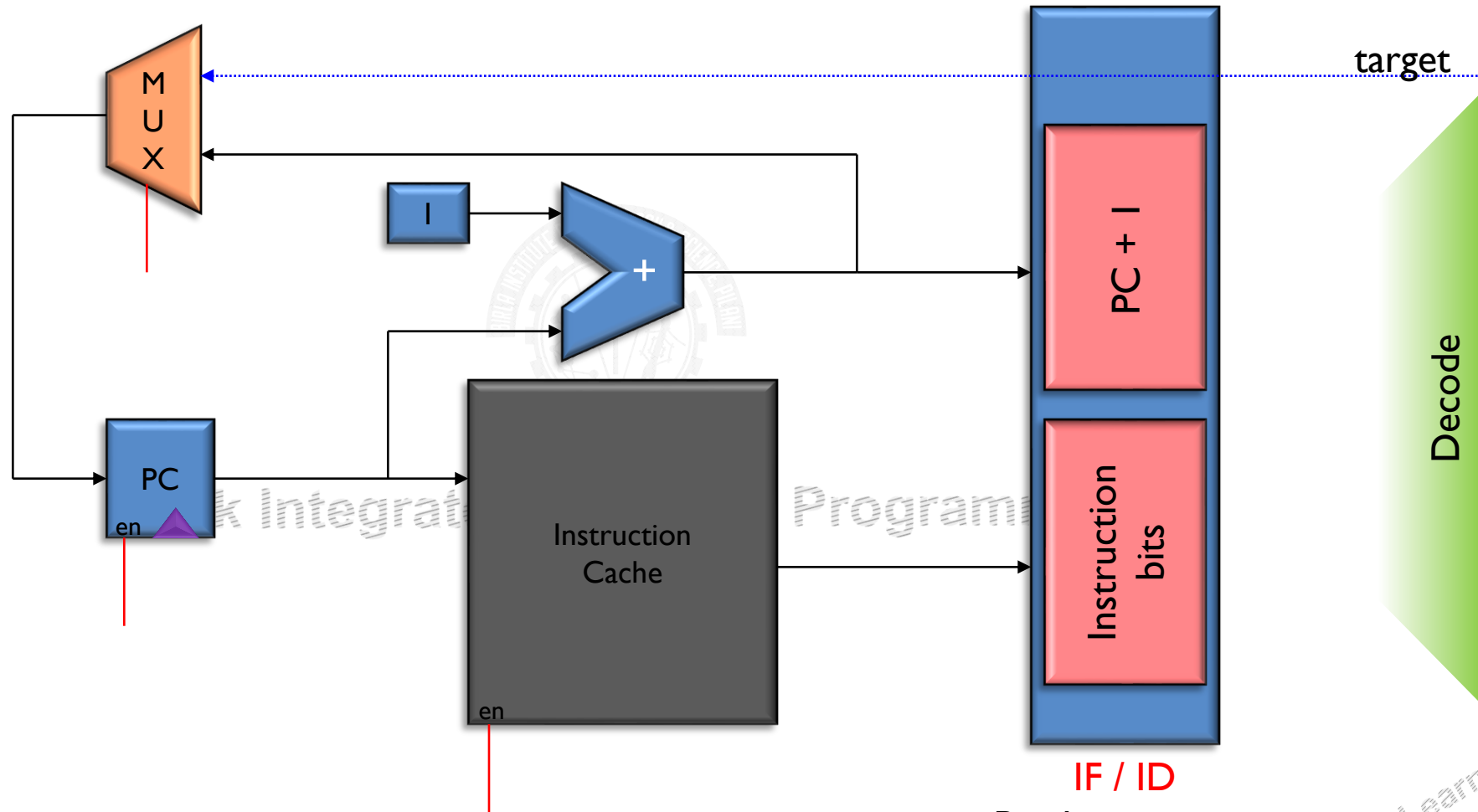
Pipeline Architecture

Processor Pipeline Review

5



Stage 1: Fetch Diagram



IF / ID

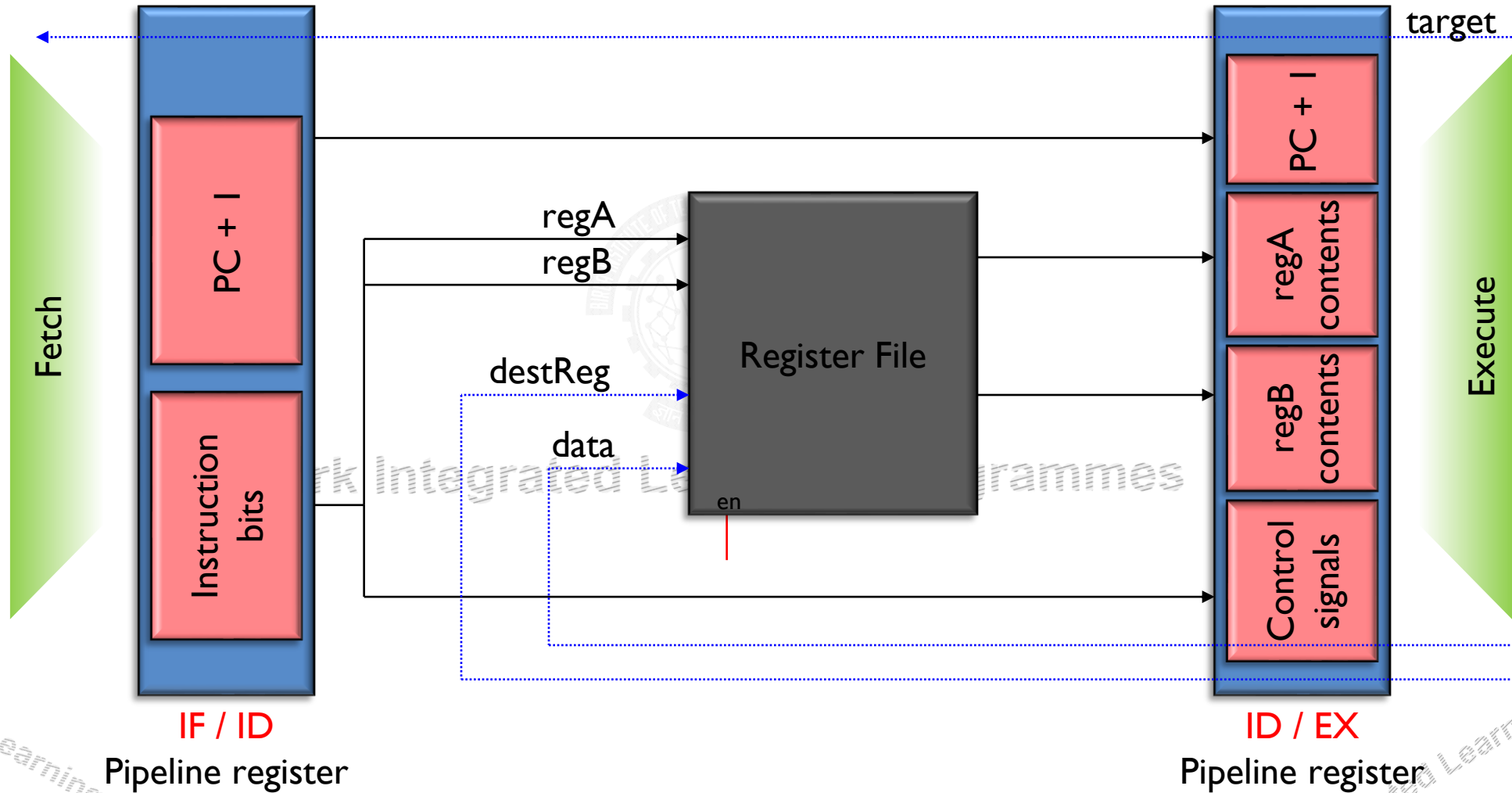
Pipeline register

3

Stage 2: Decode

- Decodes opcode bits
 - Set up Control signals for later stages
- Read input operands from register file
 - Specified by decoded instruction bits
- Write state to the pipeline register (ID/EX)
 - Opcode
 - Register contents
 - PC+1 (even though decode didn't use it)
 - Control signals (from insn) for opcode and destReg

Stage 2: Decode Diagram

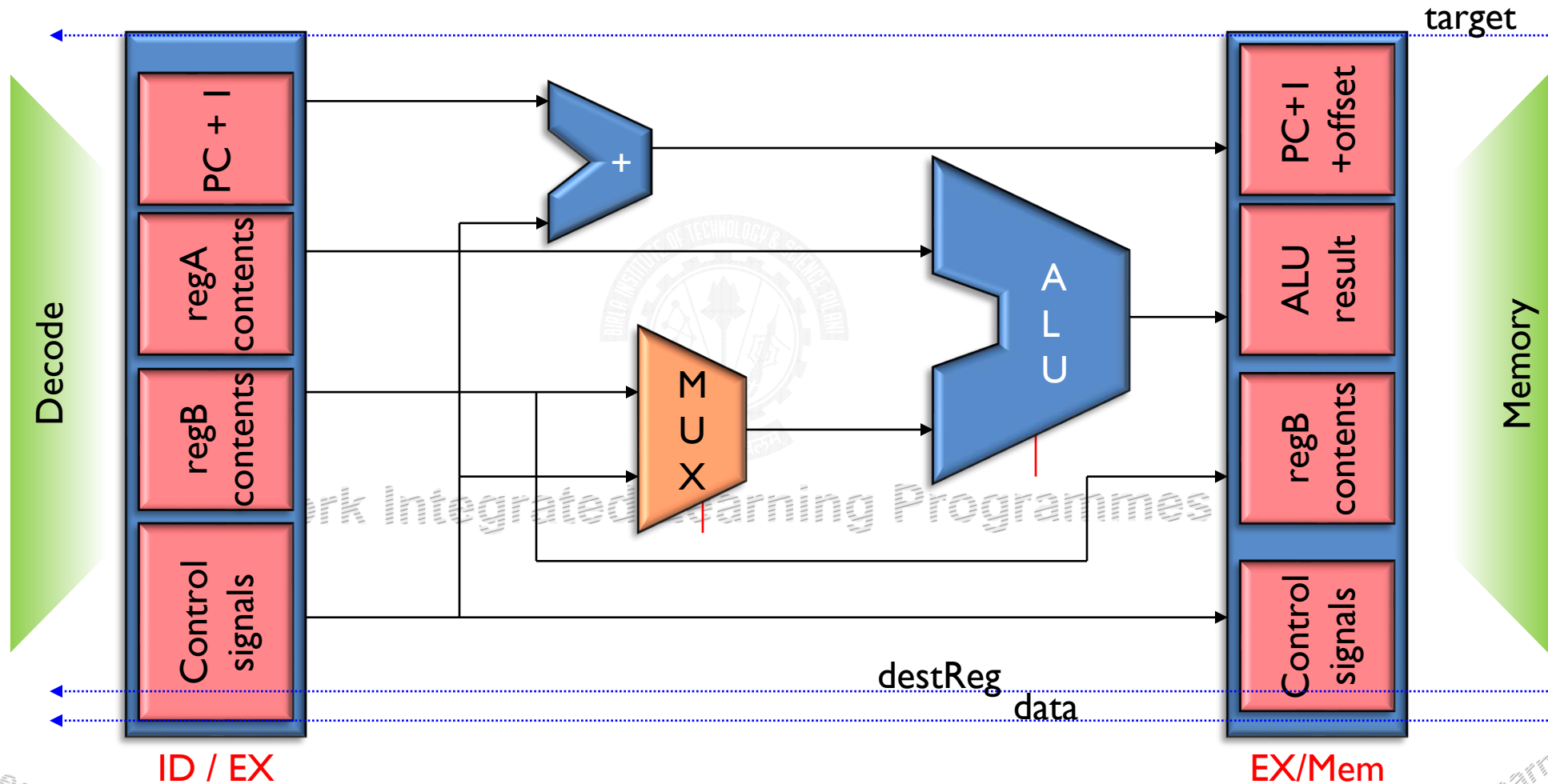


Stage 3: Execute

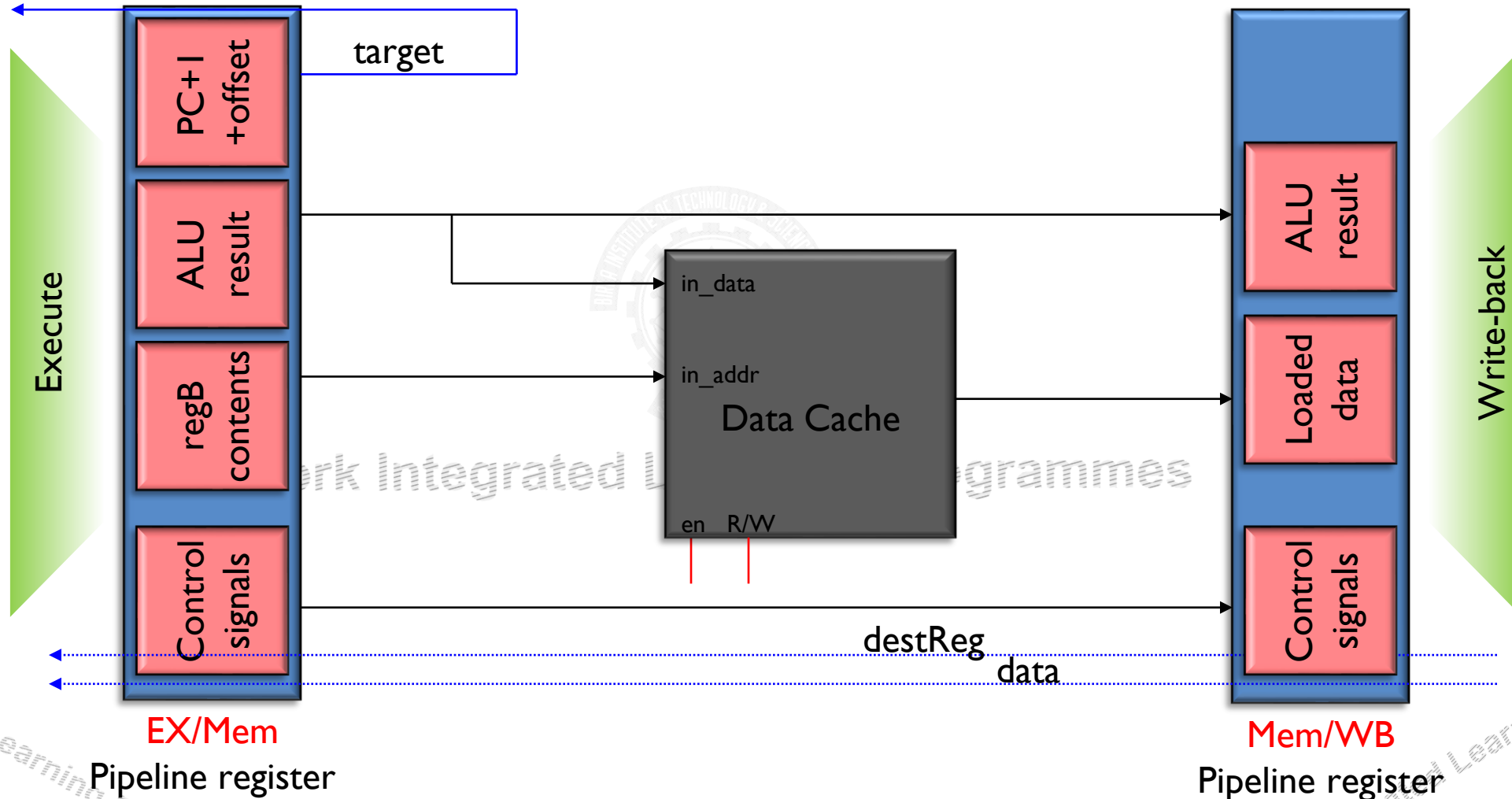


- Perform ALU operations
 - Calculate result of instruction
 - Control signals select operation
 - Contents of regA used as one input
 - Either regB or constant offset (from insn) used as second input
 - Calculate PC-relative branch target
 - $PC+1+(\text{constant offset})$
- Write state to the pipeline register (EX/Mem)
 - ALU result, contents of regB, and $PC+1+\text{offset}$
 - Control signals (from insn) for opcode and destReg

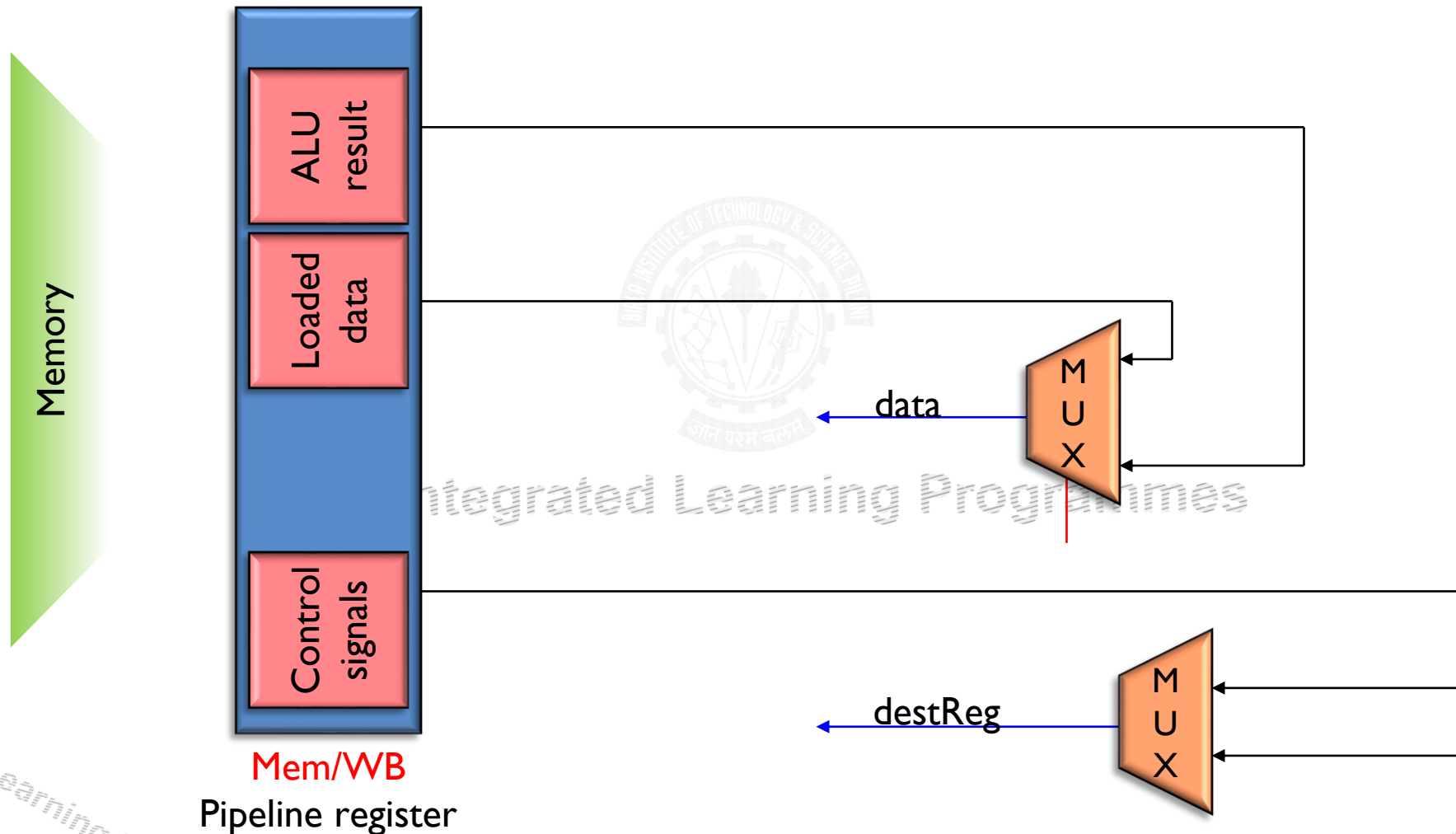
Stage 3: Execute Diagram



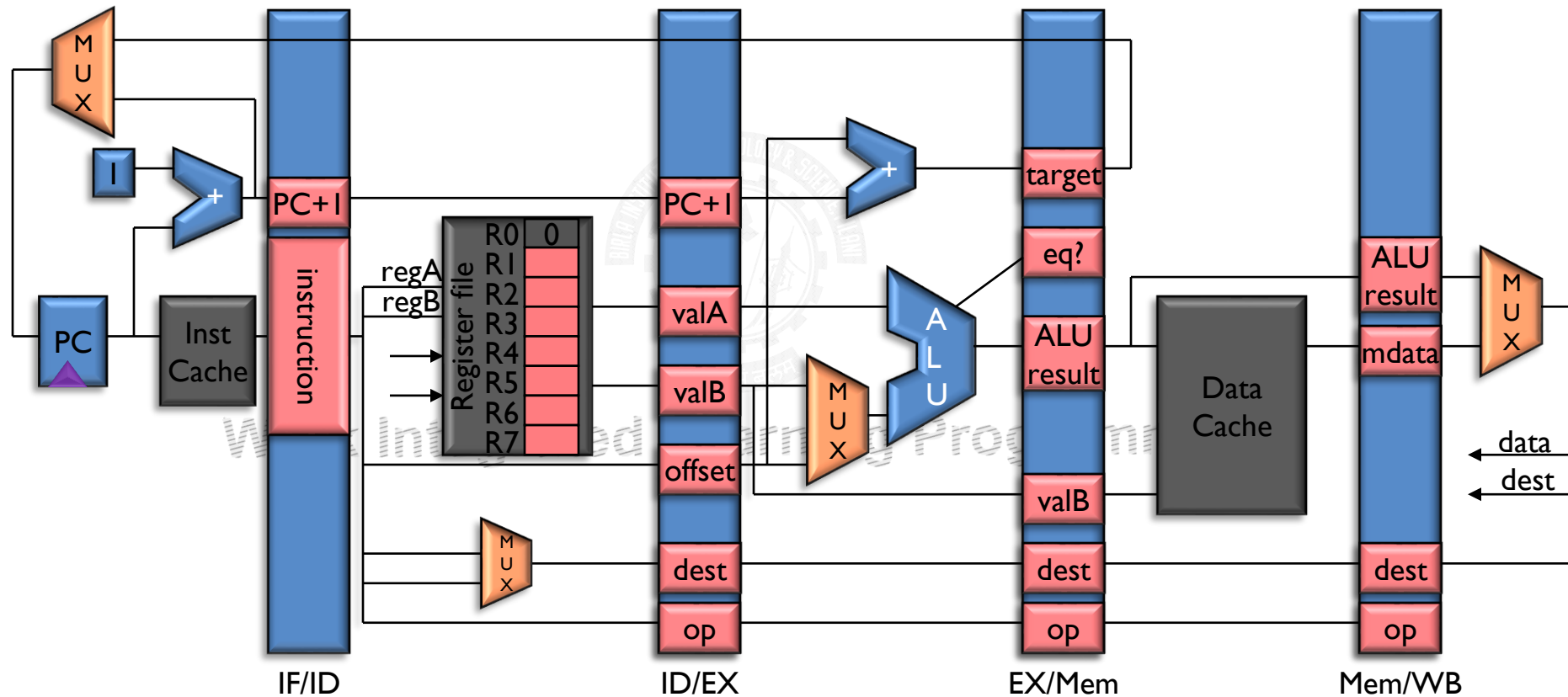
Stage 4: Memory Diagram



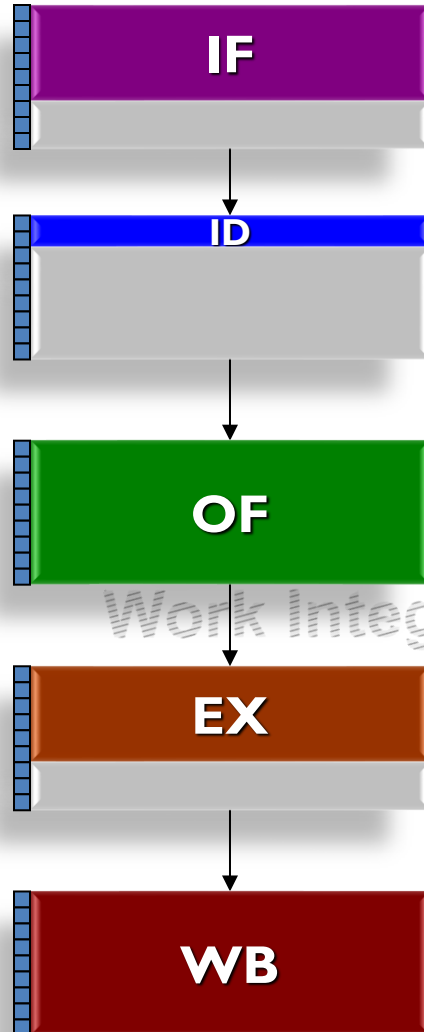
Stage 5: Write-back Diagram



Putting It All Together



Balancing Pipeline Stages



$T_{IF} = 6$ units

$T_{ID} = 2$ units

$T_{OF} = 9$ units

$T_{EX} = 5$ units

$T_{OS} = 9$ units

Without pipelining

$$T_{cyc} \approx T_{IF} + T_{ID} + T_{OF} + T_{EX} + T_{OS} \\ = 31$$

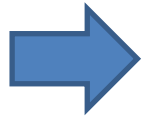
Pipelined

$$T_{cyc} \approx \max\{T_{IF}, T_{ID}, T_{OF}, T_{EX}, T_{OS}\} \\ = 9$$

$$\text{Speedup} = 31 / 9$$

Example

- If there are 20% instructions to be pipelined with 4 stage structure, what is speed up?



Using Amdahl's Law,

$$\frac{T_{\text{total}}}{T_{\text{improved}}} = \frac{1}{[T_{\text{total}} - T_{\text{component}}] + \frac{T_{\text{component}}}{n}}$$

$$\text{Speed Up} = 1 / [(1 - 0.2) + (0.2 / 4)] = 1.17$$

The percentage improvement in speed can be calculated as

$$\% \text{improvement} = 100(1 - 1/\text{Speedup}) = 15\%$$

Amdahl's Law performance

$$\frac{T_{\text{total}}}{T_{\text{improved}}} = \frac{1}{\left[\frac{T_{\text{total}} - T_{\text{component}}}{n} \right] + \frac{T_{\text{component}}}{n}}$$

N = 5

- 100% - S = 5
- 90% - S = 3.57

N = 10

- 100% - S = 10
- 90% - S = 5.26

N = 20

- 100% - S = 20
- 90% - S = 6.897

	Pipeline stages
• Flynn	2
• Foster	51
• Fishers	90

Different instructions used
Branch instruction affects the performance

• Example



- Consider the unpipelined processor. Assume that it has a 2 GHz clock (or a 0.5 ns clock cycle) and that it uses four cycles for ALU operations and branches and five cycles for memory operations. Assume that the relative frequencies of these operations are 40%, 20%, and 40%, respectively. Due to clock skew and setup, pipelining the processor adds 0.1 ns of overhead to the clock. Ignoring any latency impact, how much speedup in the instruction execution rate will we gain from a pipeline?

Solution:

1. Average instruction time on the unpipelined processor

Clock cycle = 0.5 ns.

Cycles per instruction type:

- ALU ops: 4 cycles, frequency = 40%
- Branches: 4 cycles, frequency = 20%
- Memory ops: 5 cycles, frequency = 40%

Average CPI (cycles per instruction):

$$\text{CPI}(\text{unpipelined}) = 0.4 * 4 + 0.2 * 4 + 0.4 * 5 = 1.6 + 0.8 + 2.0 = 4.4$$

Average instruction time (unpipelined):

$$T(\text{unpipelined}) = \text{CPI}(\text{unpipelined}) * \text{clock cycle} = 4.4 * 0.5\text{ns} = 2.2$$



2. Average instruction time on the pipelined processor

Pipelining adds 0.1 ns overhead to the clock, so:

$$\text{clock cycle(pipelined)} = 0.5 + 0.1 \text{ ns} = 0.6 \text{ ns}$$

Ignoring stalls and latency effects, the ideal CPI for a fully pipelined processor is 1 instruction per cycle, so:

$$T(\text{pipelined}) = 1 * 0.6 \text{ ns} = 0.6 \text{ ns}$$

3. Speedup in instruction execution rate

Speedup is the ratio of average instruction times:

$$\text{Speedup} = T(\text{unpipelined}) / T(\text{pipelined}) = 2.2 / 0.6 = 3.67$$

So, pipelining improves the instruction execution rate by roughly **3.67** over the unpipelined design, despite the 0.1 ns overhead from clock skew and setup.

Performance of Pipelines With Stalls

A stall causes the pipeline performance to degrade from the ideal performance. Let's look at a simple equation for finding the actual speedup from pipelining, starting with the formula from the previous section:

$$\begin{aligned}\text{Speedup from pipelining} &= \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}} \\ &= \frac{\text{CPI unpipelined} \times \text{Clock cycle unpipelined}}{\text{CPI pipelined} \times \text{Clock cycle pipelined}} \\ &= \frac{\text{CPI unpipelined} \times \text{Clock cycle unpipelined}}{\text{CPI pipelined} \times \text{Clock cycle pipelined}}\end{aligned}$$

Pipelining can be thought of as decreasing the CPI or the clock cycle time. Because it is traditional to use the CPI to compare pipelines, let's start with that assumption. The ideal CPI on a pipelined processor is almost always 1. Hence, we can compute the pipelined CPI:

$$\begin{aligned}\text{CPI pipelined} &= \text{Ideal CPI} + \text{Pipeline stall clock cycles per instruction} \\ &= 1 + \text{Pipelines stall clock cycles per instruction}\end{aligned}$$

If we ignore the cycle time overhead of pipelining and assume that the stages are perfectly balanced, then the cycle time of the two processors can be equal, leading to

$$\text{Speedup} = \frac{\text{CPI unpipelined}}{1 + \text{Pipeline stall cycles per instruction}}$$

One important simple case is where all instructions take the same number of cycles, which must also equal the number of pipeline stages (also called the *depth of the pipeline*). In this case, the unpipelined CPI is equal to the depth of the pipeline, leading to

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles per instruction}}$$

If there are no pipeline stalls, this leads to the intuitive result that pipelining can improve performance by the depth of the pipeline.

Pipeline Hazards



- Where one instruction cannot immediately follow another
- Types of hazards
 - Structural hazards - attempt to use the same resource by two or more instructions
 - Control hazards - attempt to make branching decisions before branch condition is evaluated
 - Data hazards - attempt to use data before it is ready
- Can always resolve hazards by waiting

• STATICALLY SCHEDULED PIPELINES



- basic pipeline: single, in-order issue
- first extension: multiple issue (superscalar)
- second extension: scheduling instructions for more Instruction Level Parallelism (ILP)
- option #1: dynamic scheduling (by the hardware)
- option #2: static scheduling (by the compiler)

• Data Hazards



- When multiple instructions are in execution simultaneously (up to five in the 5-stage pipeline), data dependencies can cause data hazards, which must be resolved for correct operation.
- Note that dependencies are properties of the software only. Data hazards are caused by data dependencies and by the structure and instruction flow of the microarchitecture.

• Data Hazards

- **RAW(read after write)** A subsequent instruction needs the result of the current instruction. If the dependent instruction does not wait for the operand it may read a stale value (a RAW hazard).

Table 3.6 RAW hazards on registers after an ALU instruction

	Clock ==>	C1	C2	C3	C4	C5	C6	C7	C8	C9
I1	ADD R1,R2,R3	IF	ID	EX	ME	WB				
I2	ADDI R3,R1,#4		IF	ID	EX	ME	WB			
I3	LW R5,0(R1)			IF	ID	EX	ME	WB		
I4	ORI R6,R1,#20				IF	ID	EX	ME	WB	
I5	SUBI R1,R1,R7					IF	ID	EX	ME	WB

• Data Hazards



- **WAR (write after read)** A subsequent instruction writes to an operand which is read by the current instruction. If the write is able to race ahead of the read, the current instruction reads a future value instead of the current value (a WAR hazard).
- **WAW (write after write)** A subsequent instruction writes into the same operand as the current instruction. If the second write is allowed to race ahead of the first write, then the final value of the operand is the first (oldest) value and therefore is stale (a WAW hazard).
- Data dependencies on memory operands do not cause hazards in the 5-stage pipeline, because all loads and stores execute in the ME stage and reach the ME stage in process order.
- There cannot be any WAW or WAR hazards on registers.
- WAW hazards are impossible because all instructions modifying the register file reach the WB stage in process order. The same argument also holds for WAR hazards because instructions always reach the decode stage, where registers are read before any subsequent instruction in process order.

• RAW/ true

- Load- use

I1: load r1, a;

I2: add r2,r1,r1;

- Define –used

I1: mul r1,r4,r5

I2:add r2,r1,r1;





- WAR /false/anti

I1: mul r1,r2,r3

I2: add r2,r4,r5



Work Integrated Learning Programmes



- WAW/output

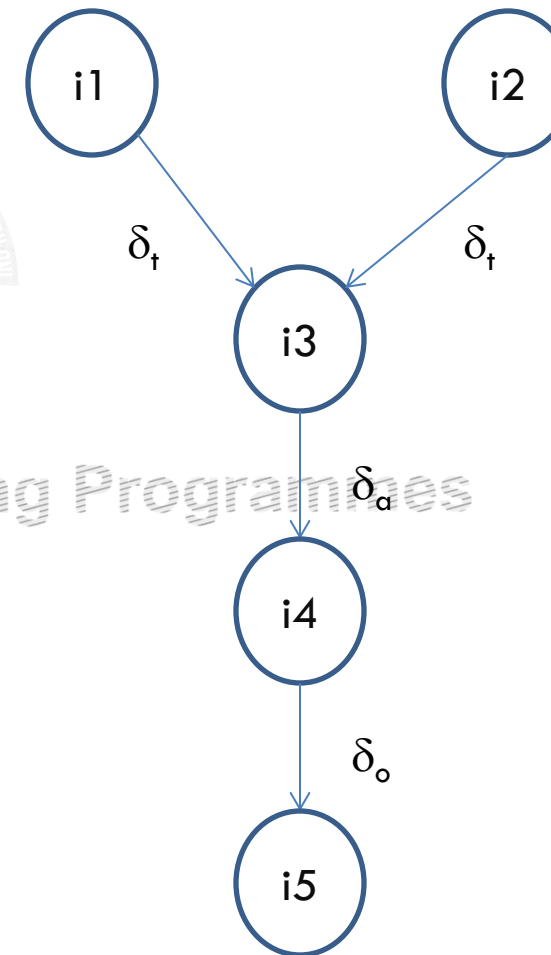
I1: mul r1,r2,r3

I2: add r1,r4,r5



Work Integrated Learning Programmes

load r1,a
 load r2,b
 add r3,r2,r1
 mul r1,r2,r4
 div r1,r2,r4



- load r1,45(r2) (1)
- add r7 $\Downarrow$ r1, r5 (2)
- sub r8 $\Downarrow$ r1, r6 (3)
- or r9 $\Downarrow$ r5, r1 (4)
- brneq r7, target (5)
- add r10 $\Downarrow$ r8, r5 (6)
- xor r2 $\Downarrow$ r3, r4 (7)
-



Work Integrated Learning Programmes

- (a) Identify each dependence; list the two instructions involved; identify which instruction is dependent; and, if there is one, name the storage location involved (register or memory).

- • Inst (2) is dependent on inst (1) through r1
- • Inst (3) is dependent on inst (1) through r1
- • Inst (4) is dependent on inst (1) through r1
- • Inst (5) is dependent on inst (2) through r7
- • Inst (6) is dependent on inst (3) through r8
- • Inst (6) is dependent on inst (5) through control
- • Inst (7) is dependent on inst (5) through control



Work Integrated Learning Programmes

- Draw a pipeline diagram for the sequence, including stalls needed to rectify the hazards. Assumptions should be mentioned properly

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
(1)	F	D	X	M	W											
(2)		F	D	d*	X	M	W									
(3)			F	p*	D	X	M	W								
(4)					F	D	X	M	W							
(5)						F	D	X	M	W						
(6)							F	D	s*							
(7)								F	s*							

In the above diagram, I assumed that the branch was predicted NT and was taken, just as an example to show the insertion of stall cycles.